Sequoia AI Ascent 2026: Software 3.0 and the Rise of Agentic Engineering

Andrej Karpathy | May 2026

https://karpathy.bearblog.dev/sequoia-ascent-2026/

Source: karpathy.bearblog.dev/sequoia-ascent-2026 — May 2026. This document reproduces the key content and verbatim quotes from Andrej Karpathy's post-talk summary of his fireside chat at Sequoia AI Ascent 2026, compiled from the original blog post and published analyses.

Overview

In May 2026, Andrej Karpathy (co-founder of OpenAI, former head of AI at Tesla, founder of Eureka Labs) spoke at Sequoia's AI Ascent conference with partner Stephanie Zhan. In a characteristically pithy move, Karpathy fed an LLM all of his recent blog posts and tweets, had it read the video transcript, and used that to generate a cleaned-up summary — which he then published as his 'Sequoia Ascent 2026' blog post. The talk introduces a three-era framework for software development and marks a critical distinction that has significant implications for how organizations build with AI.

The Software Evolution Framework: 1.0 → 2.0 → 3.0

Karpathy organizes the history of software development into three eras, each defined by how humans program computers:

Software 1.0: Humans write explicit rules in code. Logic, conditionals, algorithms. The programmer specifies exactly what the computer should do in every situation.

Software 2.0: Humans curate datasets and train neural networks. The programmer specifies the objective and provides examples; the machine learns the rules from data. This is the era of deep learning.

Software 3.0: Humans program LLMs through prompts, context, tools, examples, memory, and instructions. The 'program' is a paragraph in a context window rather than a function in a file. The LLM is the interpreter.

The Defining Insight: Specify vs. Verify

Karpathy's most important and quotable claim from the talk:

> *“Traditional computers automate what you can specify in code. This latest round of LLMs can automate what you can verify.”*

This distinction is deceptively profound. Specification requires complete, unambiguous knowledge of the procedure. Verification requires only the ability to judge whether an output is correct — a much lower bar. Almost everything humans know how to judge but can't fully articulate is now potentially automatable. This redraws the boundary between human and machine work in a way that is significantly more expansive than previous conceptions.

The Context Window as the New Programming Surface

In Software 3.0, the primary unit of programming is no longer the function — it is the context window. The LLM performs computation over the information placed in that context. This means:

The main lever is what goes into the context window: instructions, examples, memory, tool outputs, and domain knowledge. Quality of context determines quality of output. This directly validates the worldview's emphasis on context engineering as the essential competitive differentiator.

Infrastructure must become agent-native, not user-native. Most software today was designed for humans clicking buttons and reading docs. The next wave of infrastructure will be built for agents that read, act, and iterate — a fundamental redesign of the interface layer.

Vibe Coding vs. Agentic Engineering

Karpathy draws a critical distinction between two modes of AI-assisted building:

Vibe coding is when anyone builds software by describing what they want. A non-technical person creates apps, websites, tools, automations, prototypes, and internal systems by prompting an AI. It raises the floor dramatically — the barrier to creating functional software has collapsed.

Agentic engineering is what professional builders do in Software 3.0. It means using AI agents to accomplish complex, multi-step technical tasks while preserving professional quality and maintaining understanding of what is being built. It raises the ceiling for what

professionals can accomplish.

> *“Vibe coding raises the floor. Agentic engineering raises the ceiling. Both matter, and they are not the same thing.”*

December 2025: The Inflection Point

Karpathy identifies December 2025 as the moment when agentic coding tools crossed a qualitative threshold:

> *“You really had to look again as of December, because things changed fundamentally, especially in this agentic, coherent workflow. It really started to work.”*

Before December 2025, coding agents were helpful but inconsistent — they produced messy output that required significant human correction. After that point, they started producing large chunks of correct, production-quality code reliably. Karpathy himself stopped writing code manually after December, delegating entirely to agents. This is not a future prediction — it is a description of what has already occurred.

The 'Ghost' Problem and What Can't Be Outsourced

Karpathy introduces the concept of AI agents as 'ghosts' — they operate in the environment, they take actions, but their internal state and reasoning are not fully transparent. This creates a new professional requirement: mastering ghost management.

This leads to what may be the talk's most important practical warning:

> *“You can outsource thinking. You can't outsource understanding.”*

The professional who uses agentic tools to their full potential while retaining deep understanding of the domain and the system is the one who wins. The professional who delegates thinking entirely and loses the ability to verify is the one who gets replaced — not by AI, but by a human who uses AI better.

Implications for the AI Worldview

Karpathy's talk at Sequoia AI Ascent 2026 advances the worldview on several fronts:

1. Validates the context engineering thesis. The Software 3.0 framework is the technical foundation for why context engineering matters. If the context window is the program and

the LLM is the interpreter, then whoever constructs the best context wins. Domain experts who understand what belongs in that context are the scarce resource.

2. Reframes the agentic AI inflection point. December 2025 is not a prediction but a documented past event. The agentic wave the worldview identifies as '2025-2026 inflection' has already landed. The question for organizations is no longer 'when will agents work?' but 'how do we operate now that they do?'

3. Provides the 'specify vs. verify' frame. This is the clearest articulation of why AI's scope is larger than previous waves of automation. Automation has historically been limited to what humans could fully specify. The new boundary is verification — and almost every knowledge worker can verify outputs in their domain without fully being able to specify the procedure.

4. Grounds the 'You can't outsource understanding' imperative. This is the most direct statement yet of why deep domain expertise remains essential even as AI handles the execution. It is the strongest counter-argument to blanket AI replacement fears and the clearest statement of what humans must retain to remain irreplaceable.

5. Signals infrastructure redesign ahead. The shift to agent-native infrastructure is not yet complete. The next wave of value creation — for vendors, for banks, for anyone building on AI — will come from rebuilding systems designed for agents rather than bolting AI onto systems designed for humans.